

Runbook: payment-service

When to use this runbook

Open this runbook when either of these alerts fires for payment-service:

- `payment-service-p99-latency-high` (p99 > 400ms for 5 min)
- `payment-service-error-rate-high` (5xx+4xx rate > 2%)

payment-service has no user-facing endpoint of its own — it is called synchronously by checkout-api's `/payment` step (`POST /charge`). A payment-service incident is invisible to a user until they reach the payment step, and it shows up on **both** dashboards: payment-service's own metrics degrade first, and checkout-api's `/payment` p99/error rate rises in the same window as a direct consequence. Always check payment-service's own dashboard before assuming the problem is in checkout-api itself.

Triage: if `p99-latency-high` fired

1. **Check the connection pool dashboard.**

Panel: `payment-service > Postgres Pool > active\_connections / max\_connections`

- If `active\_connections` is pinned at `max\_connections` -> go to

Known Issue #1 (connection pool exhaustion).

2. **Confirm the cascade into checkout-api.**

Panel: `checkout-api > svc\_upstream\_duration\_ms{upstream="payment-service"}`

- This isolates specifically how much of checkout-api's `/payment` latency is attributable to the call into payment-service, separate from checkout-api's own processing time. If this panel's latency rose in lockstep with payment-service's own p99, the root cause is here, not in checkout-api.

3. If the above doesn't resolve it, escalate per the path below.

Triage: if `error-rate-high` fired without elevated p99 latency

1. **Break down the error rate by status code.**

Grafana panel: `payment-service > Errors by status code (4xx vs 5xx)`

2. ****If 5xx is what's climbing:****

- Check application error logs for exception stack traces on `/charge`

— a code-level bug rather than a resource-exhaustion pattern.

- Check whether checkout-api's `/payment` calls are failing with `upstream\_error` at the same rate — if so, this confirms the failures originate in payment-service and are only being observed at checkout-api, not caused by it.

Known Issue #1: Postgres connection pool exhaustion

****Symptom signature:**** p99 latency climbs gradually over 10-20 minutes, `active\_connections` pinned at `max\_connections`, application logs show `could not obtain connection from pool within 5000ms` on `/charge`.

****Immediate mitigation (no deploy required):****

- Increase the PgBouncer pool size for payment-service:

1. Edit `infra/pgbouncer/payment-pool.ini`, raise `default\_pool\_size`.

2. Reload PgBouncer without dropping existing connections:

```
psql -h <pgbouncer-host> -p 6432 pgbouncer -c "RELOAD;"
```

- Monitor `active\_connections` for 5 minutes after the change. Also watch checkout-api's `/payment` error rate — it should start recovering within a tick or two of payment-service stabilizing, confirming the cascade relationship rather than two independent issues.

Escalation path

- L1 (0-15 min): payment-service on-call, follow this runbook.

- L2 (15-30 min, unresolved): page payments-platform team lead, open incident channel `#inc-payment-latency`. Notify checkout-api on-call — their `/payment` dashboard will show correlated symptoms, and they should not independently investigate the same root cause.

- L3 (30+ min, or customer-facing revenue impact): declare SEV1, page the incident commander rotation.

Resolution criteria

- p99 latency back under 200ms for 10 consecutive minutes.
- Error rate back under 0.1%.
- checkout-api's `/payment` error rate and `svc\_upstream\_duration\_ms` for

the payment-service call both back to baseline.

- No further connection-pool alerts for 15 minutes.